



CMC UNIVERSITY

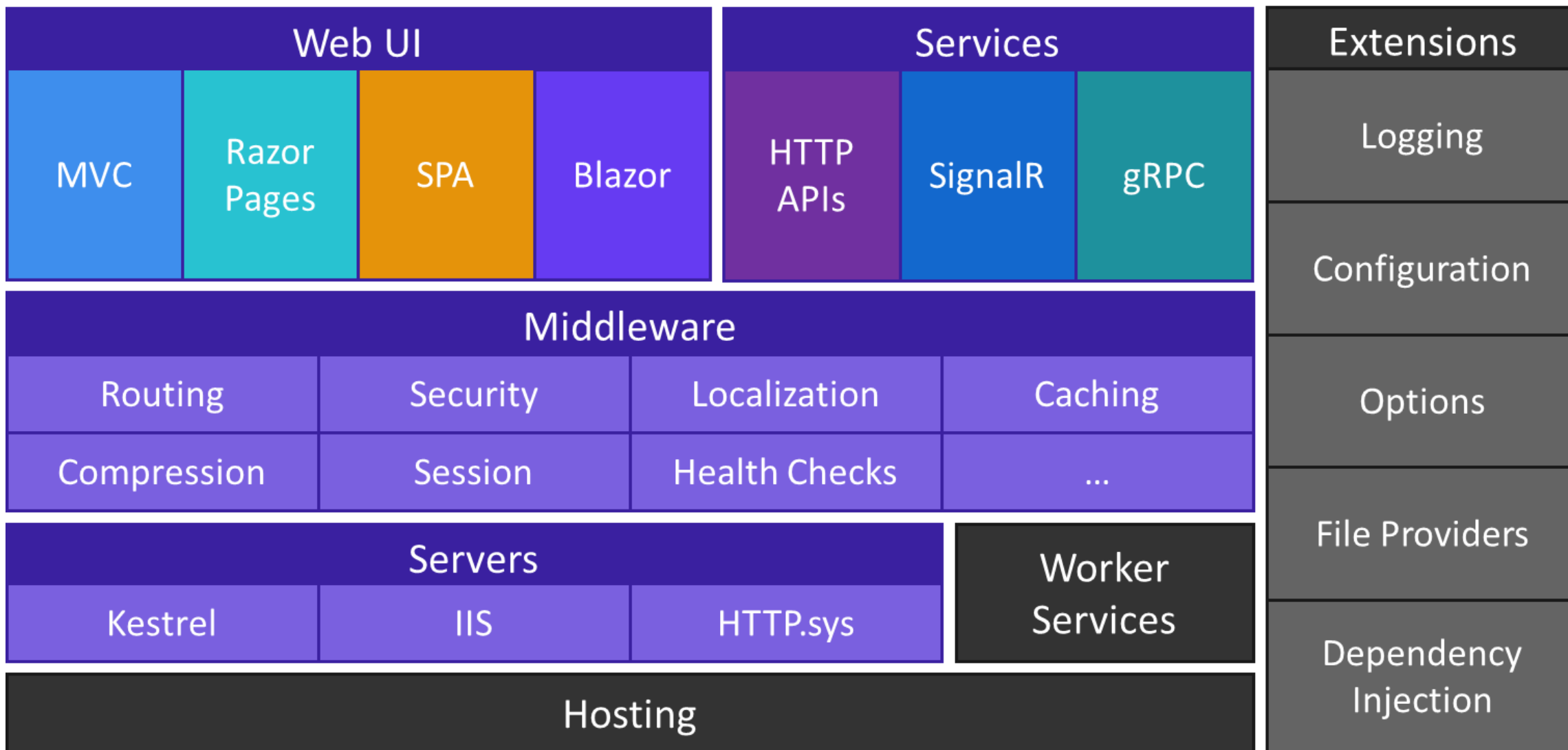
Aspire to Inspire the Digital World

ASP.NET CORE

Khoa CNTT&TT, Trường Đại học CMC

Hà Nội, tháng 12 năm 2025

ASP.NET Core là một framework mã nguồn mở, đa nền tảng (cross-platform) được phát triển bởi Microsoft, nhằm xây dựng các ứng dụng web hiện đại, API, và microservices. Đây là phiên bản cải tiến của **ASP.NET** với nhiều tính năng vượt trội về hiệu năng, khả năng mở rộng và tối ưu hóa trải nghiệm lập trình viên.



1. Hiệu năng cao

1. ASP.NET Core được tối ưu hóa để đạt hiệu năng vượt trội, đặc biệt là khi triển khai trên nền tảng cloud.
2. Sử dụng **Kestrel Web Server**, giúp tăng tốc độ xử lý và khả năng mở rộng.

2. Đa nền tảng

1. Chạy được trên Windows, macOS và Linux.
2. Hỗ trợ triển khai trên nhiều môi trường như IIS, Nginx, Docker, và Azure.

3. Modular và Lightweight (nhẹ và module hóa)

1. ASP.NET Core loại bỏ những thành phần không cần thiết từ ASP.NET Framework cũ.
2. Lập trình viên chỉ cần sử dụng các gói NuGet cần thiết, giảm tải tài nguyên.

4. Kiến trúc Middleware

ASP.NET Core sử dụng **pipeline middleware**, cho phép cấu hình và kiểm soát từng bước xử lý yêu cầu HTTP một cách linh hoạt.

5. Tích hợp Dependency Injection (DI)

DI được tích hợp sẵn, giúp quản lý các thành phần trong ứng dụng một cách dễ dàng và chuẩn hóa.

6. Nguồn mở và cộng đồng mạnh mẽ

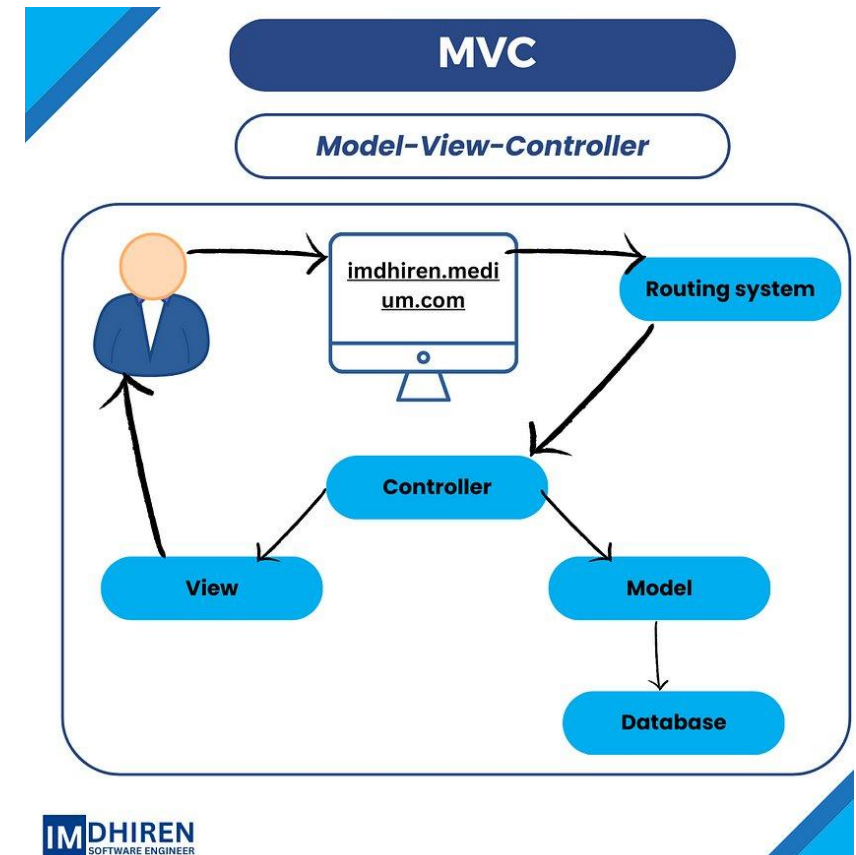
ASP.NET Core là mã nguồn mở trên GitHub, được hỗ trợ bởi một cộng đồng phát triển lớn mạnh.

- Khả năng mở rộng: Dễ dàng mở rộng khi ứng dụng phát triển hoặc tăng tải.
- Bảo mật: Hỗ trợ các tiêu chuẩn bảo mật hiện đại như OAuth, OpenID, và HTTPS.
- Hiện đại hóa: Thích hợp với các xu hướng công nghệ mới như Docker, Kubernetes và microservices.
- Phát triển linh hoạt: Hỗ trợ lập trình viên sử dụng bất kỳ hệ điều hành nào.

- Ứng dụng Web: Xây dựng các trang web từ đơn giản đến phức tạp, sử dụng mô hình **MVC** hoặc **Razor Pages**.
- **API**: Phát triển **RESTful API** để giao tiếp với các ứng dụng khác hoặc dịch vụ di động.
- Ứng dụng **Realtime**: Sử dụng **SignalR** để xây dựng ứng dụng realtime như chat, thông báo, hoặc bảng điều khiển.
- **Microservices**: Tích hợp với Docker để xây dựng hệ thống microservices phân tán.

ASP.NET MVC CORE

- ASP.NET MVC Core là một framework mạnh mẽ trong **ASP.NET Core** để xây dựng các ứng dụng web theo mô hình **Model-View-Controller**. Framework này giúp tách biệt rõ ràng các logic liên quan đến giao diện, xử lý dữ liệu, và điều khiển, giúp ứng dụng dễ bảo trì, mở rộng, và kiểm thử.



1. Tách biệt rõ ràng các thành phần:

Model, View, và Controller độc lập, giúp dễ bảo trì và mở rộng.

2. Hỗ trợ mạnh mẽ cho Unit Test:

Logic trong Controller và Model có thể được kiểm thử độc lập.

3. Tối ưu hóa SEO và RESTful API:

URL routing giúp tạo các URL thân thiện với SEO.

4. Tích hợp Dependency Injection:

Giúp quản lý các dịch vụ dễ dàng và chuẩn hóa.

Model

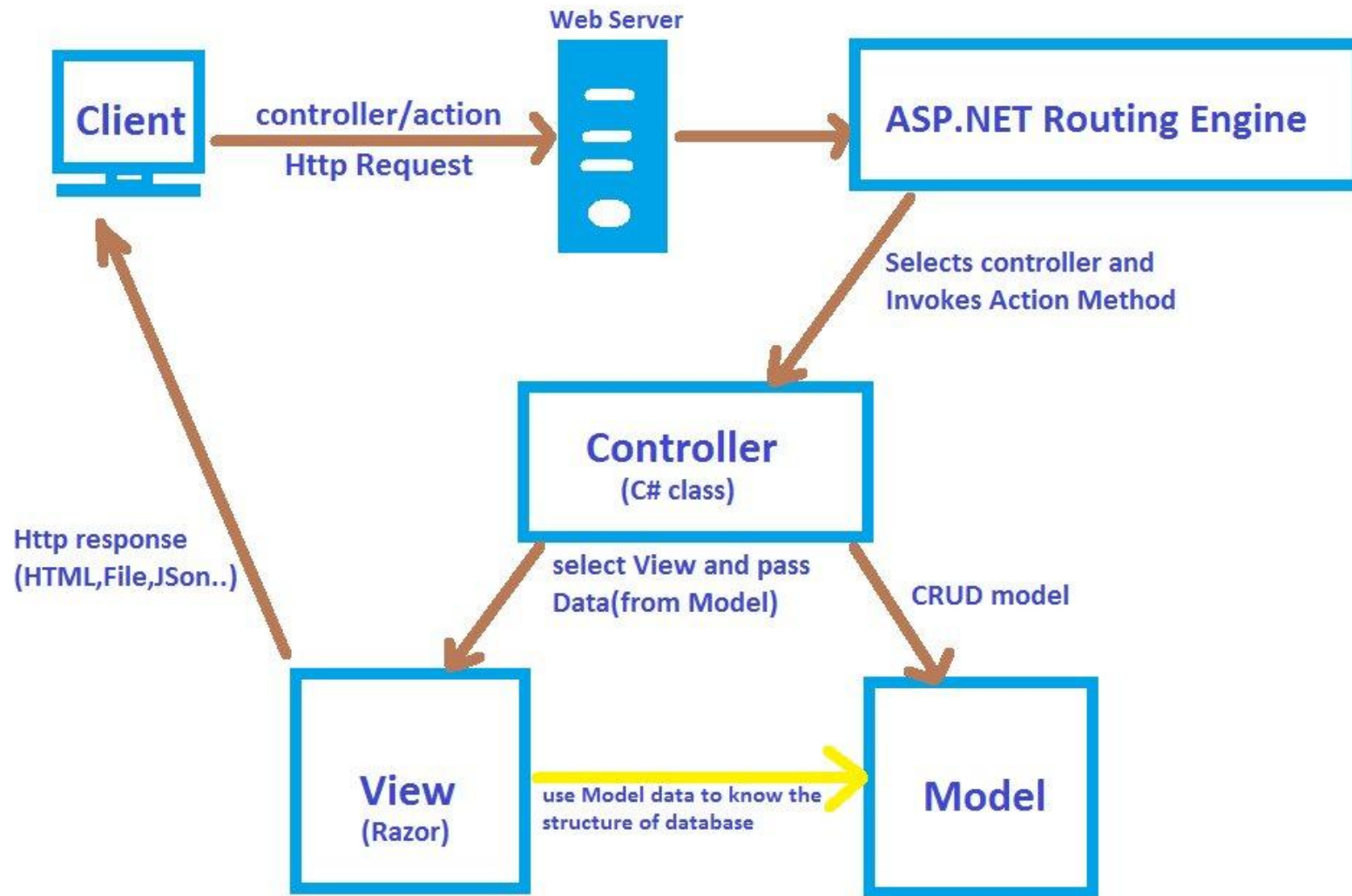
- **Vai trò:** Quản lý logic và dữ liệu nghiệp vụ.
- **Chức năng chính:**
 - Đại diện cho dữ liệu của ứng dụng.
 - Kết nối với cơ sở dữ liệu thông qua **Entity Framework** hoặc ORM khác.
 - Chứa các lớp, phương thức để thực hiện tính toán, xử lý nghiệp vụ.

View

- **Vai trò:** Hiển thị dữ liệu từ **Model** và gửi yêu cầu từ người dùng đến **Controller**.
- **Đặc điểm:**
 - Sử dụng cú pháp **Razor** để nhúng C# trong HTML.
 - Dữ liệu được truyền vào **View** thông qua các đối tượng Model.

Controller

- **Vai trò:** Nhận yêu cầu từ người dùng, xử lý logic và gửi dữ liệu đến View.
- **Chức năng chính:**
 - Điều hướng giữa Model và View.
 - Thực hiện các tác vụ xử lý trước khi gửi dữ liệu đến View.



1. Người dùng gửi yêu cầu:

Khi một URL được truy cập, yêu cầu HTTP được gửi đến ứng dụng.

2. Controller xử lý yêu cầu:

Controller nhận yêu cầu từ người dùng, thực hiện xử lý logic và lấy dữ liệu từ Model.

3. Model xử lý dữ liệu:

Nếu cần dữ liệu, Controller sẽ gọi các phương thức trong Model để lấy thông tin từ cơ sở dữ liệu.

4. View hiển thị dữ liệu:

Controller truyền dữ liệu nhận được từ Model sang View để hiển thị cho người dùng.

- ASP.NET MVC sử dụng **Routing** để điều hướng các yêu cầu HTTP đến đúng Controller và Action.

- ASP.NET MVC sử dụng Razor View Engine để nhúng mã C# vào HTML một cách linh hoạt.

- ASP.NET Core MVC hỗ trợ tích hợp sẵn Dependency Injection. Điều này giúp quản lý các dịch vụ trong ứng dụng một cách linh hoạt.



CMC UNIVERSITY

Aspire to Inspire the Digital World



Routing

Routing trong ASP.NET Core là một phần quan trọng giúp xác định cách các yêu cầu HTTP được ánh xạ đến các hành động trong Controller. Nó cung cấp sự linh hoạt trong việc định nghĩa URL cho ứng dụng và giúp xây dựng các ứng dụng RESTful dễ dàng hơn.

- Routing là gì?
- Các loại Routing
- Các tham số trong Routing
- Tích hợp Routing trong API

- **Routing** là quá trình ánh xạ một URL đến một hành động cụ thể trong Controller hoặc đến một trang cụ thể. Nó giúp định nghĩa các đường dẫn URL thân thiện với người dùng.

- Ví dụ:

URL: /products/details/5

Sẽ được ánh xạ đến

```
public IActionResult Details(int id)
{
    // id = 5
    return View();
}
```

- Routing truyền thống (Conventional Routing)
 - Ánh xạ các URL thông qua một mẫu định nghĩa trong Startup.cs.
 - Thích hợp cho các ứng dụng sử dụng mô hình MVC.

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
});
```

- URL Ví dụ:
 - / -> HomeController.Index()
 - /products/details/5 -> ProductsController.Details(int id)

- Attribute Routing
 - Sử dụng trực tiếp các **thuộc tính (attribute)** trên các Controller và Action.
 - Linh hoạt hơn so với Routing truyền thống.
 - Thích hợp cho các API hoặc đường dẫn phức tạp.

```
[Route("products")]
public class ProductsController : Controller
{
    [Route("details/{id}")]
    public IActionResult Details(int id)
    {
        return View();
    }

    [Route("list")]
    public IActionResult List()
    {
        return View();
    }
}
```

- **Tham số bắt buộc**

- Nếu không cung cấp giá trị cho tham số bắt buộc, sẽ nhận lỗi 404.

```
[Route("product/{id}")]  
public IActionResult Details(int id)  
{  
    return View();  
}
```

- URL:

- /product/5 -> Hợp lệ (id = 5)
- /product -> Lỗi 404.

- **Tham số tùy chọn:** Dùng ? để chỉ định tham số tùy chọn.

```
[Route("product/{id?}")]
public IActionResult Details(int? id)
{
    if (id == null)
        return Content("No product ID provided.");
    return Content($"Product ID: {id}");
}
```

- **URL:**
 - /product/5 -> Product ID: 5
 - /product -> No product ID provided.

- **Tham số với giá trị mặc định:** Đặt giá trị mặc định cho tham số.

```
[Route("product/{id=1}")]  
public IActionResult Details(int id)  
{  
    return Content($"Product ID: {id}");  
}
```

- URL:
 - /product/5 -> Product ID: 5
 - /product -> Product ID: 1

- Dùng để giới hạn kiểu hoặc giá trị của tham số.

```
[Route("product/{id:int}")]  
public IActionResult Details(int id)  
{  
    return Content($"Product ID: {id}");  
}
```

- **Một số ràng buộc phổ biến:**
 - {id:int}: Tham số phải là số nguyên.
 - {name:alpha}: Tham số phải là chuỗi chữ cái.
 - {id:min(1)}: Giá trị tối thiểu là 1.
 - {id:range(1,100)}: Giá trị nằm trong khoảng từ 1 đến 100.
- **URL:**
 - /product/5 -> Hợp lệ (id = 5)
 - /product/abc -> Lỗi 404.

- Routing trong **Web API** thường sử dụng **Attribute Routing** để định nghĩa các đường dẫn RESTful.

```
[Route("api/[controller]")]
[ApiController]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok(new[] { "Product1", "Product2" });
    }

    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
        return Ok($"Product ID: {id}");
    }
}
```

- URL Ví dụ:
 - GET /api/products -> Lấy tất cả sản phẩm.
 - GET /api/products/5 -> Lấy sản phẩm với ID = 5.



CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU